Football Blind Draft — Architecture and Protocols

MTM4681 Mobile Programming · Milestone 1 · D6 · Flutter (iOS + Android) · Pattern: MVVM

1. Design pattern: Model–View–ViewModel (MVVM)

We use **MVVM** because Flutter rebuilds widgets from state: the View should only draw, a ViewModel per screen owns the screen state and reacts to user actions, and the Model holds the game rules as plain Dart that can be unit-tested without a device. Services sit behind interfaces so the same ViewModels work with a fake (offline, tests, bot practice) and a Firebase implementation (online 1v1).

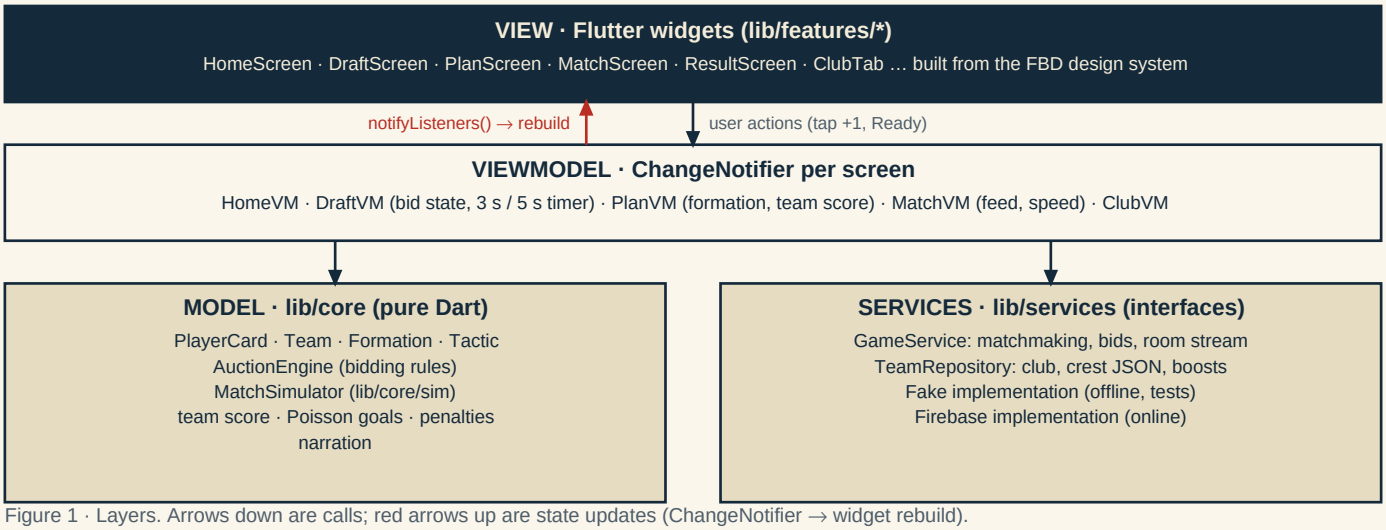

Figure 1 · Layers. Arrows down are calls; red arrows up are state updates (ChangeNotifier → widget rebuild).

2. How each part applies to the app

Part	In Football Blind Draft	Example
Model (lib/core)	Plain Dart classes and rules, no Flutter imports: PlayerCard (overall, ATK, PAS, DEF, position, style), Team, Formation, Tactic; AuctionEngine; MatchSimulator in lib/core/sim.	AuctionEngine knows the 3 s opener window, the 5 s window that resets on every bid, that the last bidder cannot bid twice and that an unopened card goes free to the opener.
View (lib/features)	Stateless widgets built from the FBD design system (cards, countdown, bid buttons, pitch). They read the ViewModel and forward taps.	DraftScreen shows the Countdown, the Player Card and the three bid buttons; disabled state comes from the ViewModel.
ViewModel	One ChangeNotifier per screen; holds UI state, timers and validation; calls services; exposes simple values to the View.	DraftViewModel: current card, balances, leader, remaining seconds, canBid. PlanViewModel: formation, slot assignments, team score and chemistry preview.
Services (lib/services)	Interfaces for everything outside the device; fake and Firebase implementations are swapped at start-up.	GameService.watchRoom(roomId) returns a Stream of RoomState that DraftViewModel listens to.

3. Key interfaces and their purpose

Interface	Responsibility	Main methods
GameService	Online match lifecycle: queue, room, bids, plan lock, result.	joinQueue(), cancelQueue(), watchRoom(id) → Stream<RoomState>, placeBid(id, amount), pass(id), lockPlan(id, plan)
AuctionEngine	Pure bidding rules used by the server and for UI validation (same code, same result).	open(card, opener), bid(team, amount) → Result, tick(now), close() → winner, price
MatchSimulator	Team score (position penalty, chemistry, tactic), Poisson goals, penalties, narration events.	teamScore(team), simulate(home, away, seed) → MatchResult(events, score, penalties)
TeamRepository	Persistent club data: name, code, crest JSON {shape, pattern, kit, icon, abbr}, keeper boosts, record.	loadClub(), saveClub(club), saveBoosts(ids), history()
PlayerRepository	Fictional player pool (assets/data/players.json) and draft pool generator (12 cards, tier mix).	allPlayers(), generatePool(seed)

4. Protocols

Protocol / technology	Used for	Why
HTTPS (REST-style Cloud Functions)	placeBid, pass, lockPlan, getResult	The server is the referee: it checks timers, balances and rules so a modified client cannot cheat.
Firestore real-time listeners (gRPC/WebSocket streaming)	Room state: current card, bids, leader, timer deadline, squads, match events	Both players see a bid within a fraction of a second without polling.
Firebase Authentication (ID token / JWT)	Anonymous sign-in at first launch; token sent with every request	Identifies the player without a sign-up form; the server knows who bid.
JSON	Room documents, crest {shape, pattern, kit, icon, abbr}, player data	Readable, maps directly to Dart classes with fromJson / toJson.
Server time	Bid windows use the server deadline, not the phone clock	Keeps 3 s / 5 s windows fair on slow or wrong-clock devices.

5. Sequence: a player places a bid

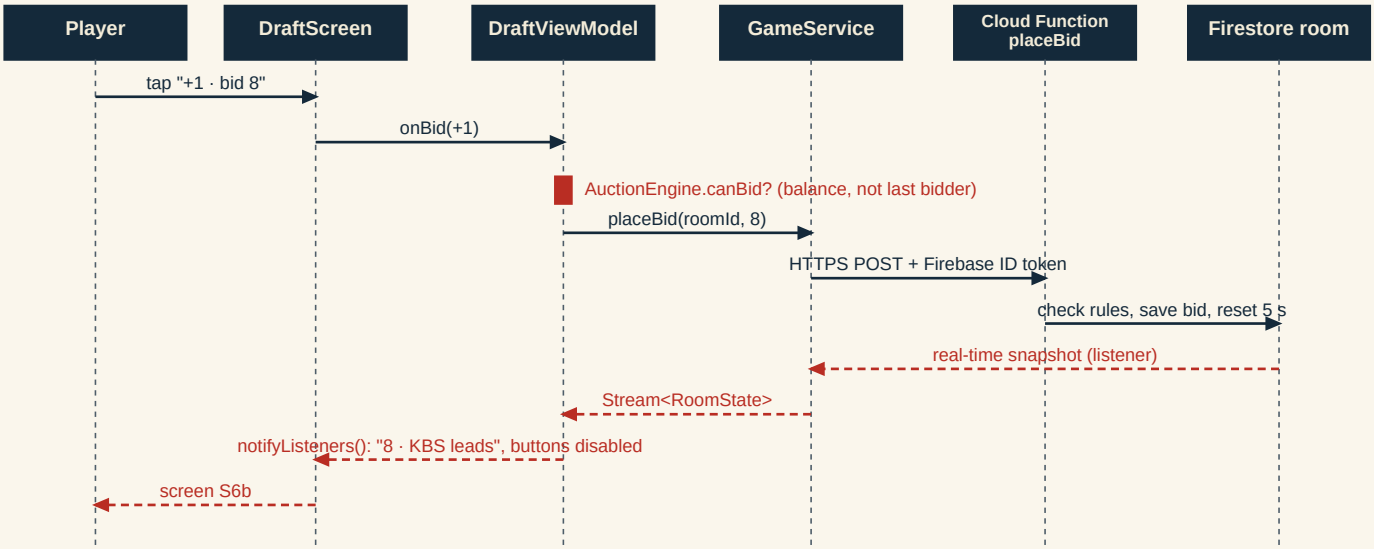

Figure 2 · The ViewModel validates locally for instant feedback, but the Cloud Function decides. Both players' screens update from the same Firestore snapshot.

6. Folder structure

`lib/app` navigation and theme (design tokens from Figma variables) · `lib/core` models and AuctionEngine · `lib/core/sim` MatchSimulator · `lib/features/{draft, plan, match, team}` screens and ViewModels · `lib/services` GameService and repositories (fake + Firebase) · `test/` unit tests for AuctionEngine and MatchSimulator (e.g. a 10 000-match calibration test: a 5-point stronger team wins about 70 % of matches).

Prepared with AI assistance (Claude), see README.